

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 6-6
Student Name: D. Vijaya Sri
Roll No.: A24126552262

1. TITLE

Node.js Modules — **os**, **path** and **fs**

2. AIM

To understand Node.js modules and demonstrate the use of the built-in **os**, **path**, and **fs** modules for operating-system information, file-path handling, and file operations.

3. OBJECTIVE

The objectives of this experiment are:

- To understand modularity in Node.js.
- To use the built-in **os** module to obtain system information.
- To use the **path** module to construct and manage file paths.
- To use the **fs** module to create, write, and read a file.
- To display and verify the results through the Node.js terminal.

4. THEORY

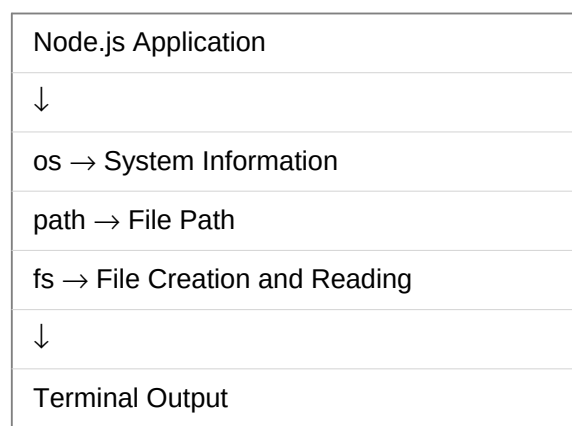
Node.js modules are reusable pieces of functionality that can be imported into a Node.js application. Node.js provides several built-in modules that can be used without installing external packages.

os module: The **os** module provides information about the operating system and hardware. In this experiment, **os.platform()**, **os.arch()**, **os.hostname()**, and **os.cpus()** are used.

path module: The **path** module provides utilities for working with file and directory paths. **path.join()** is used to create a valid path independent of manual path separators.

fs module: The **fs** module provides file-system operations. **fs.writeFileSync()** writes data to a file and **fs.readFileSync()** reads data from a file.

Basic flow:



5. ALGORITHM / PROCEDURE

1. Create a Node.js project folder.
2. Create an **app.js** file.
3. Import the built-in **os**, **path**, and **fs** modules.
4. Obtain operating-system information using the **os** module.
5. Create the path of **student.txt** using the **path** module.
6. Write student information into the file using the **fs** module.
7. Read the file using **fs.readFileSync()**.
8. Display the system information, path information, and file contents in the terminal.
9. Verify that the file was created successfully and that its contents were read correctly.

6. PROGRAM

app.js

```
// Import built-in Node.js modules
const os = require("os");
const path = require("path");
const fs = require("fs");

// Display operating system information
console.log("===== OPERATING SYSTEM INFORMATION =====");
console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());
console.log("Hostname:", os.hostname());
console.log("Number of CPUs:", os.cpus().length);

// Create a file path using the path module
const filePath = path.join(__dirname, "student.txt");

console.log("\n===== PATH INFORMATION =====");
console.log("File path:", filePath);

// Data to write into the file
const studentData = `
Student Name: Vijayasri
Course: B.Tech CSE (AI & ML)
Subject: Full Stack Web Development
Topic: Node.js Modules
`;

// Write data into the file
fs.writeFileSync(filePath, studentData);

console.log("\nFile created successfully.");

// Read the file
const fileContent = fs.readFileSync(filePath, "utf8");

console.log("\n===== FILE CONTENT =====");
console.log(fileContent);
```

Expected project structure:

```
Q6-NodeJS-Modules/
├──
├── app.js
├── student.txt
```

7. CODE EXPLANATION

Code / Method	Explanation
<code>require("os")</code>	Imports the built-in OS module.
<code>require("path")</code>	Imports the path module for file and directory paths.
<code>require("fs")</code>	Imports the file-system module.
<code>os.platform()</code>	Returns the operating-system platform.
<code>os.arch()</code>	Returns the system architecture.
<code>os.hostname()</code>	Returns the computer hostname.
<code>os.cpus().length</code>	Returns the number of CPUs reported by Node.js.
<code>path.join()</code>	Creates a valid file path from path components.
<code>fs.writeFileSync()</code>	Creates or overwrites a file and writes data to it.
<code>fs.readFileSync()</code>	Reads the contents of a file synchronously.
<code>console.log()</code>	Displays information in the terminal.

8. TEST CASES AND EXECUTION DATA

The program was executed successfully and the observed terminal output was used for verification.

Test Case	Test / Input	Expected Result	Actual Result	Status
TC-01	Run node app.js	OS information should be displayed.	Platform, architecture, hostname and CPU count displayed.	Pass
TC-02	Execute path handling	File path should be generated.	Path to student.txt displayed.	Pass
TC-03	Execute file creation	File should be created successfully.	"File created successfully."	Pass
TC-04	Read created file	File contents should be displayed.	Student, course, subject and topic displayed.	Pass

9. OUTPUT

Output 1: Node.js Execution

```
c:\Users\dadhi\FULL_STACK\01-observation-and-record\observation\Task 6\Q6-NodeJS-Modules\src>node app.js
===== OPERATING SYSTEM INFORMATION =====
Platform: win32
Architecture: x64
Hostname: VIJAYASRI
Number of CPUs: 8

===== PATH INFORMATION =====
File path: c:\Users\dadhi\FULL_STACK\01-observation-and-record\observation\Task 6\Q6-NodeJS-Modules\src\student.txt

File created successfully.

===== FILE CONTENT =====

Student Name: Vijayasri
Course: B.Tech CSE (AI & ML)
Subject: Full Stack Web Development
Topic: Node.js Modules

c:\Users\dadhi\FULL_STACK\01-observation-and-record\observation\Task 6\Q6-NodeJS-Modules\src>
```

9. OUTPUT

Observed execution details

Item	Observed Value
Platform	win32
Architecture	x64
Hostname	VIJAYASRI
Number of CPUs	8
File path	...\Q6-NodeJS-Modules\src\student.txt
File operation	File created successfully.
File content	Student Name: Vijayasri; Course: B.Tech CSE (AI & ML); Subject: Full Stack Web Development; Topic: Node.js Modules

10. RESULT

Thus, the **os**, **path**, and **fs** built-in modules of Node.js were successfully demonstrated. The program retrieved operating-system information, generated a file path, created **student.txt**, and read its contents successfully. The execution was verified using the actual terminal output.